

PROGRAMMING FUNDAMENTALS

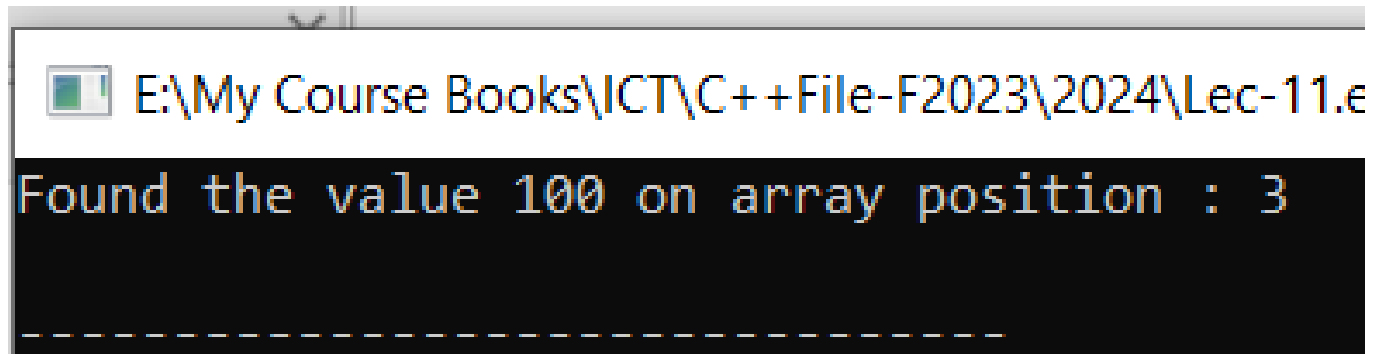
Spring 2024

28th March

Linear Search in an Array

```
#include <iostream>
using namespace std;
int main()
{
    const int SIZE = 5;
    int num[SIZE] = {87, 75, 98, 100, 82};
    int n = 100;
    int i = 1;
    int found = 0;
    while(i < SIZE && found != 1)
    {
        if(num[i] == n)
        {
            found = 1;
            break;
        }
        i = i + 1;
    }
    if (found == 0)
        cout << "did not find value " << n << " in the array\n";
    else
        cout << "Found the value " << n << " on array position : " << i << endl;
    return 0;
}
```

Output..



```
E:\My Course Books\ICT\C++File-F2023\2024\Lec-11.e
Found the value 100 on array position : 3
-----
```

The image shows a screenshot of a Windows command prompt window. The title bar at the top reads "E:\My Course Books\ICT\C++File-F2023\2024\Lec-11.e". The main area of the window has a black background with white text. The first line of text is "Found the value 100 on array position : 3". Below this line is a dashed line consisting of approximately 20 hyphens.

Linear Search

- It is simple
- It is very easy to understand and implement
- It doesn't require the data in the array to be stored in any particular order
- BUT If the array contains 20,000 elements, the algorithm will have to look at all 20,000 elements in order to find a value stored in the last element

Linear Search

- When the linear search fails to locate an item, it must make a comparison with every element in the array
- As the number of failed search attempts increases, so does the average number of comparisons
- → the linear search should not be used on large arrays if the speed is important

The Binary Search

- It is much more efficient than the linear search
- It requires the values in the array be sorted in some order
- Instead of testing the array's first element, this algorithm starts with the element in the middle
- If middle element is the desired value, then the search is over
- Otherwise, the value in the middle element is either greater than or less than the value being searched for.

Binary Search

- If the middle value is greater, then the desired value (if it is in the list) will be found somewhere in the first half of the array
- If it is less, then the desired value (again, if it is in the list) will be found somewhere in the last half of the array
- → half of the array's elements have been eliminated from further searching

Binary Search Code

```
#include <iostream>
using namespace std;
int main()
{
    int SIZE = 20;
    // Array with employee IDs sorted in ascending order.
        int idNums[SIZE] = {101, 142, 147, 189, 199, 207, 222, 234,
289, 296, 310, 319, 388, 394, 417, 429, 447, 521, 536, 600};
        int empID; // To hold an employee ID
    // Get an employee ID to search for
    cout <<"Enter the employee ID you wish to search for: ";
    cin>>empID;
    int first = 0, // First array element
    last = SIZE - 1; // Last array element
    int middle; // Midpoint of search
    bool found = false; // Flag
    int position;
```



```
while (!found && first <= last)
{
    middle = (first + last) / 2; // Calculate midpoint
    if (idNums[middle] == emplID) // If value is found at mid
    {
        found = true;
        position = middle;
    }
    else if (idNums[middle] > emplID) // If value is in lower half
        last = middle - 1;
    else
        first = middle + 1; // If value is in upper half
    }
    // If position contains -1 the emplID was not found.
    if (position == -1)
        cout << "That emplID does not exist in the array. \n";
    else
    {
        // Otherwise position contains the subscript of
        // the specified employee ID in the array.
        cout << "That ID is the element number " << position+1;
        cout << " in the array.\n";
    }
    return 0;
}
```

Output..

 E:\My Course Books\ICT\C++File-F2023\2024\Lec-11.exe

```
Enter the employee ID you wish to search for: 600  
That ID is the element number 20 in the array.
```

 E:\My Course Books\ICT\C++File-F2023\2024\Lec-11.exe

```
Enter the employee ID you wish to search for: 555  
That empID does not exist in the array.
```

Why Binary Searching

- More efficient than the linear search
- Every time it makes a comparison and fails to find the desired item, it eliminates half of the remaining portion of the array that must be searched
- Consider an array with 1,000 elements. If the binary search fails to find an item on the first attempt (at middle position), the number of elements that are required to be searched is 500
- If the item is not found on the second attempt (2nd Middle position), the number of elements that remains to be searched is 250
- Process continues until the target element is either located or is declared not present in the array

Binary or Linear

- With 1,000 elements, this takes no more than 10 comparisons
- Using linear we will require an average of 500 comparisons

Powers of 2 method (for Binary Search)

- To calculate the maximum number of comparisons the binary search will need with an array of any size
- Simply find the smallest power of 2 that is **greater than or equal to** the number of elements in the array
- For example, a maximum of 16 comparisons will be required if we have an array of 50,000 elements
- $2^{16} = 65,536$
- Maximum of 20 comparisons will be required on an array of 1,000,000 elements
- $2^{20} = 1,048,576$

Sorting an array

Why Sorting?

- The data in an array is mostly sorted in some order
- Student Lists, for instance, are commonly sorted in alphabetical order or with RegNo
- Student grades sorted from highest to lowest
- Product list sorted according to price

Sorting Algorithms

- A sorting algorithm is a technique for scanning through an array and rearranging its contents in some specific order

Sorting Order

- Ascending order means the values in the array are stored from lowest to highest
- If the values are sorted in descending order, they are stored from highest to lowest

1. The Bubble Sort

- As the larger values drop to the bottom of the array, the smaller values rise (like bubbles) to the top
- Called **Bubble Sort**

Bubble Sort

- Algorithm provides a quick and easy way to sort the items stored in an array, as long as the number of items is relatively small—for example, fewer than 50.
- It works by comparing adjacent array elements and interchanging (swapping) the ones that are out of order
- The algorithm continues comparing and swapping until the data in the array is sorted

Bubble Sorting explained

7	2	3	8	9	1
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

2	7	3	8	9	1
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

2	3	7	8	9	1
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

2	3	7	8	1	9
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

2	3	7	1	8	9
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

FINALLY...

1	2	3	7	8	9
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

Selection Sort

- The selection sort performs fewer exchanges because it moves items immediately to their final position in the array
- The smallest value in the array is located and moved to element 0
- Then the next smallest value is located and moved to element 1
- This process continues until all of the elements have been placed in their proper order

Selection Sort

5	7	2	8	9	1
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

1	7	2	8	9	5
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

1	2	7	8	9	5
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

Selection Sort

1	2	5	8	9	7
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

1	2	5	7	9	8
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

1	2	5	7	8	9
Element 0	Element 1	Element 2	Element 3	Element 4	Element 5

Ex-1

- Write code to accept user input for the size of an integer array. Then ask user to enter values in the array. Display the original array and then swap the first and the last element of the array. Display the updated array after the swap operation.

Output..

E:\My Course Books\ICT\C++File-F2023\2024\Lec-12.exe

Enter the Size of the Array = 5

Enter the Array values

2

50

6

10

800

Thanks for entering the values in the array

The Array entered by the user

2

50

6

10

800

The Array After the swap Operation

800

50

6

10

2

Ex-2

- Create an integer array of size 9 by using the following formula:
- $\text{MyArray}[i] = (i*2)+5$
- Display the array after filling in the values
- Multiply each element of the array by 3
- Display the array after changing it using a separate loop

Ex-3

- Define two arrays that may be used in parallel to store the ten employee identification numbers (as integers) and gross pay amounts (as doubles).
- Initialize these using the declaration statements.
- Write a loop that uses these arrays to print each employee's identification number and weekly gross pay

Ex-4

- Write a program that lets the user enter the total rainfall for each of 12 months into an array of doubles.
- The program should calculate and display the total rainfall for the year and the average monthly rainfall.

2D Arrays

- Several identical arrays put together
- To store multiple sets of data

Application

- For example, in a grade-averaging program a teacher might record all of one student's test scores in an array
- If the teacher has 5 students and 3 tests, that would require 15 variables to store scores of all tests for all 5 students.
- It would be better to define a two-dimensional array
- This 2 D array can hold multiple sets of data in rows and columns

2D Array

	Column 0	Column 1	Column 2	Column 3
Row 0	scores[0] [0]	scores[0] [1]	scores[0] [2]	scores[0] [3]
Row 1	scores[1] [0]	scores[1] [1]	scores[1] [2]	scores[1] [3]
Row 2	scores[2] [0]	scores[2] [1]	scores[2] [2]	scores[2] [3]

- The above array has three rows (numbered 0 through 2), and four columns (numbered 0 through 3)
- It has a total of 12 elements in the array

Declaring 2D array

- Two size indices are required
- The first one is for the number of rows
- The second one is for the number of columns

```
int num[3][4];
```

- Declares an integer array with three rows and four columns
- each element has two subscripts: one for its row and another for its column

```
cout << num[0][2];
```

- displays the array element at row 0, column 2:

Book Reference

- D.S. Malik (ch - 8)
- Tony Gaddis (ch - 7)